SÉANCE 3 SUR 3

PROGRAMMATION ORIENTÉE OBJET — JAVA

Types Génériques et Collections

Du code réutilisable avec typage sûr et des structures de données concrètes mobilisant les séances précédentes

À LA FIN DE CETTE SÉANCE

Objectifs pédagogiques

À la fin de cette séance, vous serez capable de…

1. Expliquer l'intérêt des génériques et écrire une classe ainsi qu'une méthode génériques.
2. Utiliser des paramètres de type bornés et des wildcards, notamment la règle PECS.
3. Choisir et justifier l'utilisation d'ArrayList ou LinkedList, HashSet ou TreeSet, HashMap ou TreeMap.
4. Parcourir une Map et la combiner avec Comparable et Iterator pour résoudre un problème simple.
5. Combiner génériques, collections, comparaison, héritage et interfaces dans une solution cohérente.

LIEN AVEC LA SÉANCE 2

Le socle construit en séance 2

- Comparable et Comparator définissent l'ordre des objets.
- Iterator et Iterable organisent le parcours sans exposer la représentation interne.
- Composite combine interfaces, collections et récursion.
- Cette séance explique enfin les types génériques List<T> et Map<K,V>, puis guide le choix des collections.

AVANT / APRÈS

Pourquoi la programmation générique ?

```
// AVANT les génériques — tout est Object
List names = new ArrayList();
names.add("Ana");
names.add(42);                          // compile malgré
l'erreur de type
String s = (String) names.get(1);     // ClassCastException
à l'EXÉCUTION

// APRÈS les génériques — le compilateur vérifie
List<String> names2 = new ArrayList<>();
names2.add("Ana");
names2.add(42);                         // erreur de
COMPILATION — détectée immédiatement
String s2 = names2.get(0);              // aucun cast
nécessaire
```

- **Les génériques déplacent les erreurs de type de l'exécution vers la compilation.**
- Plus besoin de casting manuel lors de la lecture d'une collection.
- C'est pourquoi les collections utilisées depuis la séance 2 sont écrites List<X>, et non simplement List.

Q4.1

Définir une classe générique

```
public class Pair<T> {
    private T first;
    private T second;

    public Pair(T first, T second) {
        this.first = first;
        this.second = second;
    }
    public T getFirst()  { return first; }
    public T getSecond() { return second; }
}

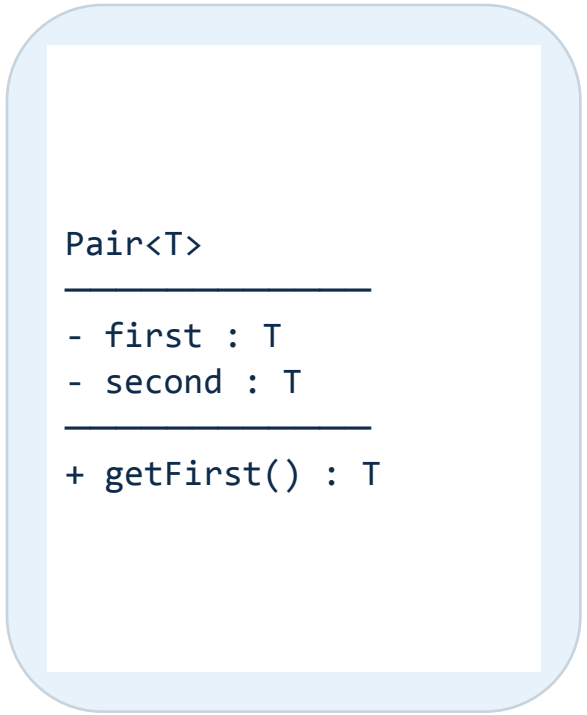
Pair<String> names = new Pair<>("Ana", "Leo");
Pair<Integer> scores = new Pair<>(18, 20);
```

- **T est un paramètre de type : un emplacement remplacé par un type réel lors de l'utilisation.**
- Pair<String> et Pair<Integer> sont deux instanciations différentes de la même classe.
- Convention : une lettre majuscule — T (type), E (élément), K/V (clé/valeur).

Les paramètres de type font partie du modèle

Une classe générique est représentée avec son paramètre de type.

Le même symbole T doit rester cohérent dans les attributs, les paramètres et les types de retour.





```
class Pair<T> {
  private T first;
  private T second;

  T getFirst() {
    return first;
  }
}
```

UN PARAMÈTRE DE TYPE SUR LA MÉTHODE, PAS SUR LA CLASSE

Q4.2

Méthodes génériques

```
public class Utils {
    public static <T> T first(List<T> list) {
        return list.get(0);
    }
}

String s = Utils.first(List.of("Ana", "Leo"));   // T
déduit comme String
Integer i = Utils.first(List.of(1, 2, 3));        // T
déduit comme Integer
```

- **<T> placé avant le type de retour déclare le paramètre de type propre à la méthode.**
- Il est indépendant des éventuels paramètres de type de la classe.
- Le compilateur déduit T depuis l'argument ; il est rarement nécessaire de l'écrire explicitement.

INVARIANCE

Les types génériques sont invariants afin de préserver la sûreté du typage

Integer est un sous-type de Number.

Cependant, List<Integer> n'est pas un sous-type de List<Number>.

Sinon, un code recevant List<Number> pourrait insérer un Double dans une liste censée ne contenir que des Integer.



LIMITER LES TYPES ACCEPTÉS PAR T

Q4.3

Paramètres de type bornés

```
public class Box<T extends Number> {
    private final T value;
    public Box(T value) { this.value = value; }
    public double doubled() { return value.doubleValue() *
2; }
}
Box<Integer> b1 = new Box<>(21);    // OK — Integer extends
Number
Box<String>  b2 = new Box<>("texte");    // erreur de
compilation — String n'est pas un Number

// bornes multiples : classe d'abord, puis interfaces
public class Ranked<T extends Number & Comparable<T>> { /*
... */ }
```

- **T extends Number limite T à Number ou à une de ses subclasses et rend accessibles les méthodes de Number, comme doubleValue().**
- Un T non borné (<T>) se comporte comme Object : aucune méthode supplémentaire n'est disponible.
- Bornes multiples : au maximum une classe, obligatoirement en premier, puis des interfaces.

Wildcards et PECS

PECS — Producer Extends, Consumer Super — le moyen mnémotechnique standard pour choisir une wildcard.

Wildcard	Signification	À utiliser lorsque…
List<?>	type inconnu	seules des méthodes indépendantes du type sont nécessaires, par ex. size()
List<? extends Number>	Number ou un sous-type — producer	vous lisez uniquement la liste : extends produit les valeurs consommées
List<? super Integer>	Integer ou un supertype — consumer	vous écrivez uniquement des Integer : super consomme les valeurs produites

PECS : Producer Extends, Consumer Super — lire depuis un producer et écrire vers un consumer.

Q4.4

Utilisez extends pour lire et super pour écrire

Une wildcard représente une famille de types génériques liés.

? extends T produit des valeurs pouvant être lues comme T en toute sécurité.

? super T consomme des valeurs pouvant être écrites comme T en toute sécurité.

Producer : ? extends Number

↓

Lire les valeurs comme Number en toute sécurité

↓

Consumer : ? super Integer
écrire des Integer en toute sécurité

Ce que l’on peut lire et écrire

PECS indique l’usage sûr, mais ne transforme pas une liste en un type plus général.

Avec extends, on lit comme Number sans connaître le sous-type exact. Avec super, on peut ajouter un Integer, mais une lecture n’est garantie que comme Object.

```
List<? extends Number> src
  Number n = src.get(0);
  // src.add(3) interdit
```

↓

```
List<? super Integer> dst
      dst.add(3);
  Object x = dst.get(0);
```

↓

extends → lire
super → écrire

Restrictions des génériques

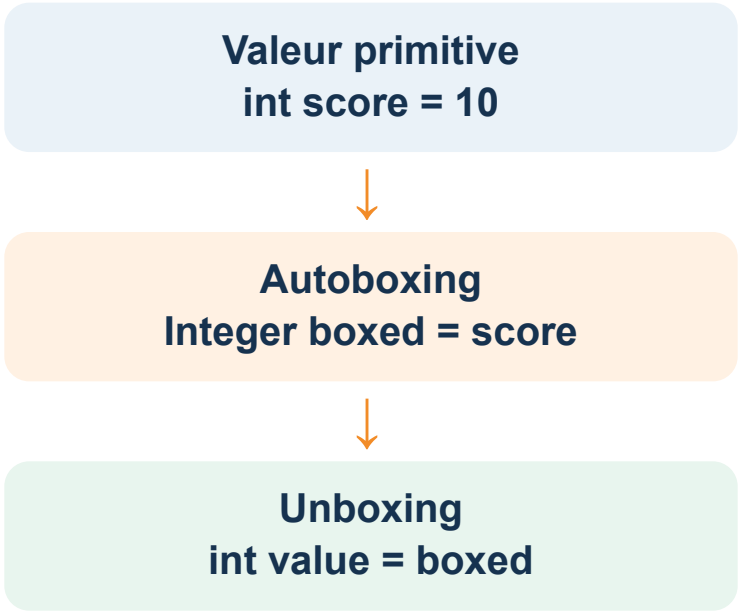
- Pas de new T() : la JVM efface les informations génériques à l'exécution ; T est donc inconnu lors de la création de l'objet.
- Pas de tableaux génériques : new T[10] ne compile pas ; utilisez par exemple ArrayList<T>.
- Pas de type primitif comme paramètre : List<int> est interdit ; utilisez le wrapper List<Integer>.
- Un membre static ne peut pas utiliser le paramètre de type de la classe : static appartient à la classe avant le choix de T.

L’autoboxing relie les primitifs aux types génériques par référence

Les arguments de type génériques doivent être des types par référence ; List<int> est donc interdit.

Les classes wrapper représentent les valeurs primitives sous forme d’objets : Integer, Double, Boolean, etc.

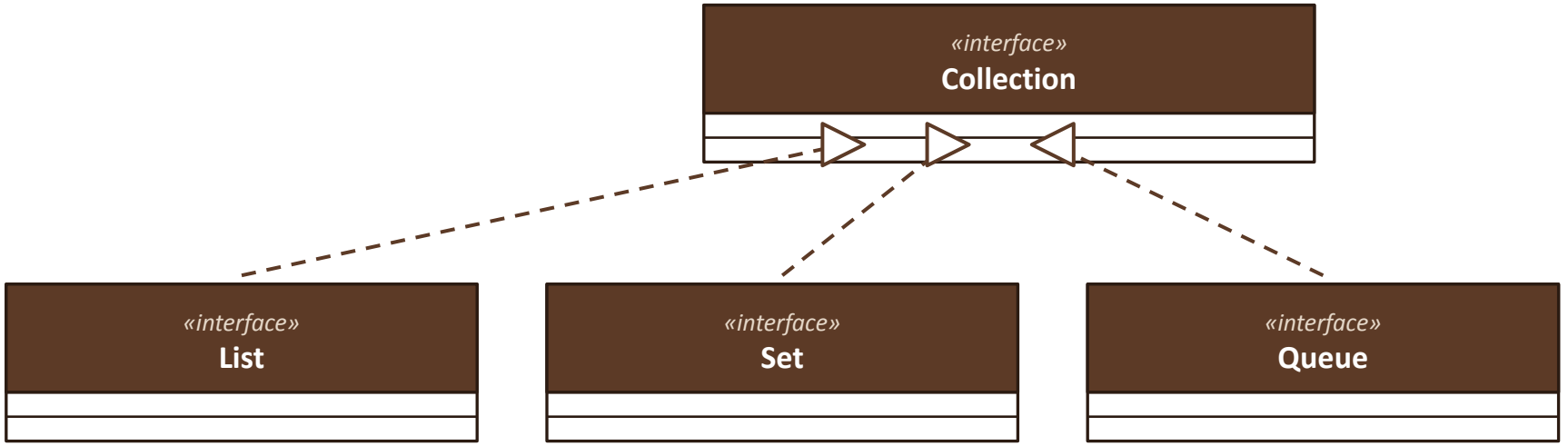
Java insère automatiquement les conversions boxing et unboxing lorsque le contexte est non ambigu.



Le framework Collections

List, Set et Map : les structures concrètes au cœur de la plupart des programmes à venir.

Collections Framework — Vue d'ensemble



Map<K,V> appartient à une hiérarchie distincte : elle contient des paires et non des éléments isolés ; ce n'est donc pas une Collection.

List — ordonnée, doublons autorisés, accès par indice (ArrayList, LinkedList)

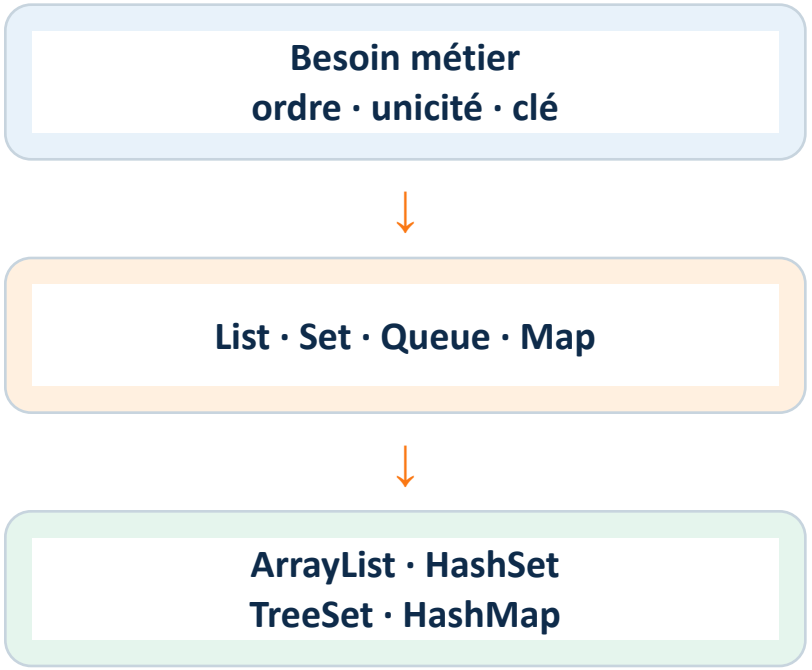
Set — aucun doublon (HashSet, TreeSet)

Queue — éléments en attente de traitement, généralement FIFO (LinkedList, PriorityQueue)

Choisir le contrat avant l’implémentation

Commencez par le comportement attendu, puis consultez la hiérarchie JDK.

L’interface exprime le contrat ; la classe concrète détermine notamment les performances et l’ordre.



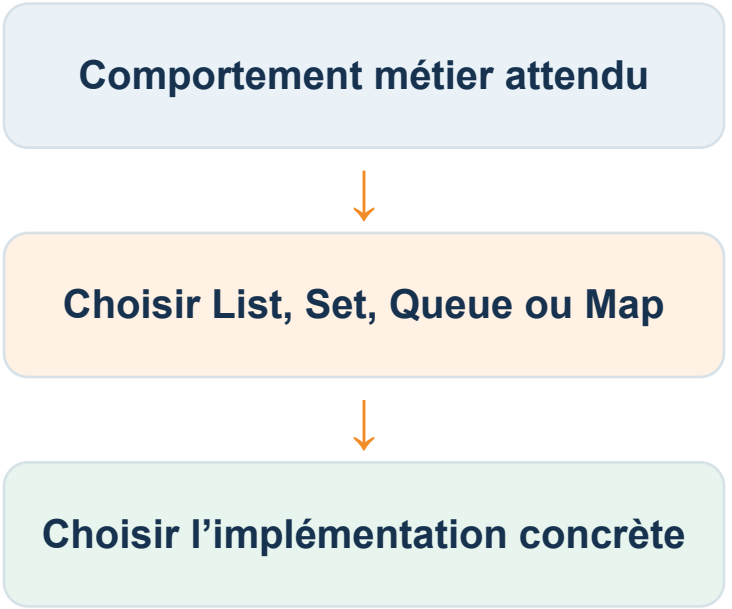
JDK 21 · Collection

Choisir d’abord l’interface, puis l’implémentation

Programmez selon le comportement nécessaire.

List, Set, Queue et Map décrivent des contrats différents.

ArrayList, HashSet et HashMap sont des choix d’implémentation qui pourront évoluer.



Interface Collection — Rappel

- add(E e) / remove(Object o) — ajouter ou supprimer un élément.
- size() / isEmpty() — connaître le nombre d'éléments et vérifier si la collection est vide.
- contains(Object o) — tester la présence d'un élément.
- iterator() — retourne un Iterator<E>, comme vu en séance 3 ; il alimente les boucles for-each.
- Une variable est rarement déclarée directement en Collection<E> : choisissez une interface plus précise, List, Set ou Queue.

Q4.5

List : ArrayList ou LinkedList

	ArrayList	LinkedList
Structure interne	tableau redimensionnable	chaîne de nœuds doublement liés
get(index) — accès aléatoire	rapide	lent (parcours de la chaîne)
ajout à la fin	rapide en général	rapide
insertion / suppression au milieu	lent (décalage des éléments)	rapide une fois la position atteinte
Idéal pour…	lectures fréquentes par indice	insertions/suppressions fréquentes, peu d'accès par indice

Règle pratique : en cas de doute, commencez par ArrayList ; c'est le bon choix par défaut dans la plupart des cas.

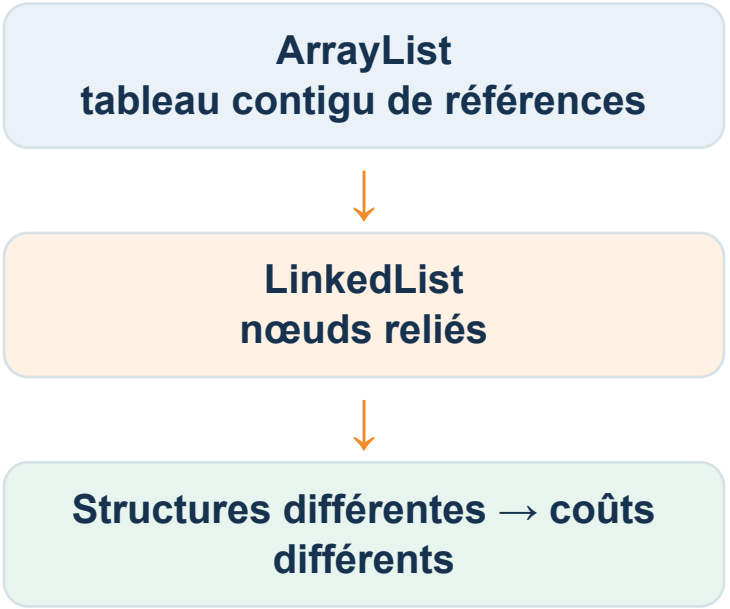
Q4.5

La structure interne explique les performances

ArrayList stocke les références dans un tableau contigu redimensionnable.

LinkedList stocke des nœuds reliés dans les deux directions.

Chaque modèle interne rend certaines opérations plus ou moins coûteuses.



AUCUN DOUBLON — MAIS DANS QUEL ORDRE ?

Set : HashSet ou TreeSet

```
Set<String> fast = new HashSet<>();
fast.add("banana"); fast.add("apple"); fast.add("banana");
// fast contient 2 éléments — ordre d'itération non
spécifié

Set<String> sorted = new TreeSet<>();
sorted.add("banana"); sorted.add("apple");
sorted.add("cherry");
// itération dans l'ordre naturel : apple, banana, cherry
// TreeSet exige des éléments Comparable ou un Comparator
```

- **HashSet : add/contains/remove en O(1) attendu avec une bonne distribution du hash ; aucun ordre d'itération garanti.**
- TreeSet : reste trié en permanence selon l'ordre naturel ou un Comparator ; légèrement plus lent, il utilise Comparable vu en séance 3.
- La détection des doublons dépend d'un contrat equals()/hashCode() correct pour HashSet, ou de compareTo() pour TreeSet.

RECHERCHE CLÉ → VALEUR

Map : HashMap ou TreeMap

Q4.6

```
Map<String, Integer> fast = new HashMap<>();
fast.put("apple", 3);
fast.put("banana", 5);
int n = fast.get("apple");              // 3
boolean has = fast.containsKey("kiwi");  // false

Map<String, Integer> sorted = new TreeMap<>();
sorted.put("banana", 5);
sorted.put("apple", 3);
// keySet() / entrySet() parcourent les clés dans l'ordre
croissant
```

- **put(key, value), get(key) et containsKey(key) sont les opérations essentielles.**
- HashMap est rapide et non ordonnée ; TreeMap reste triée en permanence selon ses clés.
- Une Map n'est pas une Collection : elle gère des paires et possède sa propre interface, distincte de List/Set/Queue.

Q4.6

Parcourir une Map

```
Map<String, Integer> prices = new TreeMap<>();
prices.put("bread", 2);
prices.put("milk", 1);

for (Map.Entry<String, Integer> entry : prices.entrySet())
{
    System.out.println(entry.getKey() + " -> " +
entry.getValue());
}

for (String key : prices.keySet()) {
    System.out.println(key + " coûte " + prices.get(key));
}
```

- **entrySet() est la manière idiomatique d'obtenir ensemble clé et valeur, sans appels répétés à get().**
- keySet() et values() fournissent uniquement les clés ou les valeurs lorsque c'est suffisant.

METTRE LES NOTIONS ENSEMBLE

Choisir la bonne collection

1

Avez-vous besoin de paires clé → valeur ?

→ Map — HashMap (rapide) ou TreeMap (triée par clé)

2

Chaque élément doit-il être unique ?

→ Set — HashSet (rapide) ou TreeSet (trié)

3

Avez-vous besoin d'un accès par indice ou d'un ordre maîtrisé ?

→ List — ArrayList (par défaut) ou LinkedList (insertions/suppressions fréquentes)

4

Voulez-vous traiter les éléments dans leur ordre d'arrivée ?

→ Queue — ArrayDeque par défaut, ou PriorityQueue si une priorité est requise

MODÈLE DE DÉCISION

Le comportement attendu détermine la collection

Partez du besoin métier, pas du nom d’une classe familière.

Interrogez-vous sur la recherche clé-valeur, l’unicité, l’ordre, l’indexation et le tri.

Choisissez ensuite l’interface la plus simple qui garantit ces propriétés.



Exemple guidé : compter les occurrences de mots

```
public static Map<String, Integer> countWords(String[]
words) {
    Map<String, Integer> counts = new TreeMap<>();
    for (String w : words) {
        counts.put(w, counts.getOrDefault(w, 0) + 1);
    }
    return counts;
}

countWords(new String[]{"java", "oop", "java", "java",
"oop"});
// {java=3, oop=2} — trié par clé grâce à TreeMap
```

- **getOrDefault(key, 0) rend explicite le pattern « lire zéro si absent, puis incrémenter ». merge est une variante idiomatique à découvrir ensuite.**
- C’est le même raisonnement que pour construire un index ou une structure groupée ; vous le retrouverez rapidement.
- Remplacez TreeMap par HashMap si le tri des clés n’est pas nécessaire.

RAPPEL AVANT LE TP2

Génériques + collections — Vue d'ensemble

- List<Employee> est préférable à une List raw pour la même raison que Box<Integer> est préférable à une Box raw : le compilateur contrôle les entrées et les sorties.
- La classe utilitaire Collections fournit des opérations prêtes à l'emploi : sort, reverse, max et min, utilisant Comparable ou Comparator vus en séance 3.
- Toutes les structures étudiées peuvent contenir vos propres objets si equals()/hashCode() sont correctement implémentées et, lorsque nécessaire, Comparable.

SYNTHÈSE

À retenir

- ✓ Les génériques détectent les erreurs de type à la compilation : List<T>, méthodes génériques et types bornés T extends X.
- ✓ Wildcards : ? extends pour lire (producer), ? super pour écrire (consumer) — retenez PECS.
- ✓ List, Set et Map proposent des compromis différents entre vitesse et ordre selon leur implémentation.
- ✓ entrySet() est la méthode standard pour parcourir une Map avec la clé et la valeur.
- ✓ Toutes les notions des séances 1 à 3 — encapsulation, héritage, interfaces, exceptions, génériques et collections — convergent dans le TP2.